Web Development I

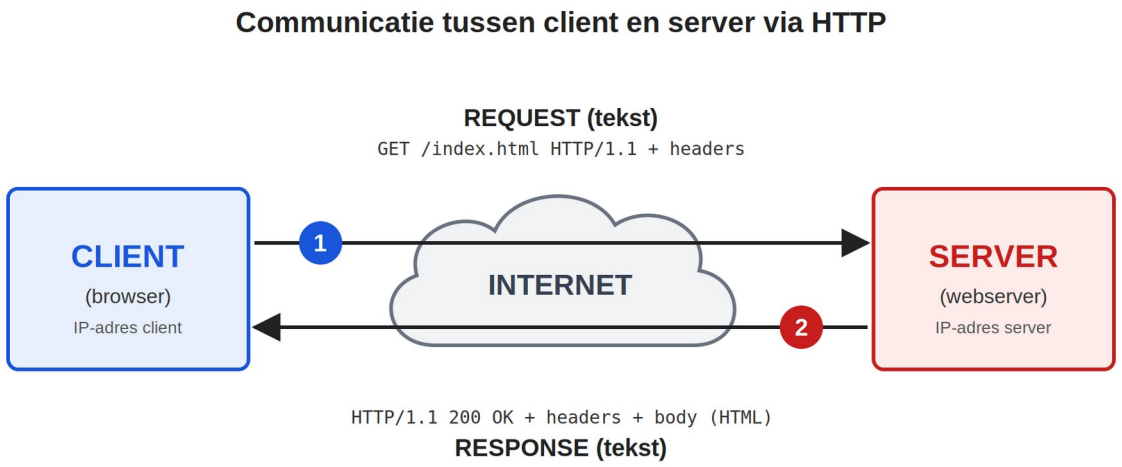
Labo 1 – Internet en HTTP

Verslag

Matei Lucian Aldea
Hogeschool VIVES
29 september 2026

Opdracht 1

De client (browser) stuurt een request naar de server. Dat bericht is tekst en bevat de URL van de resource die de browser wil ontvangen. Het bericht gaat via het internet naar de server. De server stuurt een response terug naar het IP-adres van de client. Die response is ook tekst en bevat een statuscode, headers en de inhoud (body), bijvoorbeeld een HTML-document.



Figuur 1 – Schema van de communicatie tussen client en server

Opdracht 2

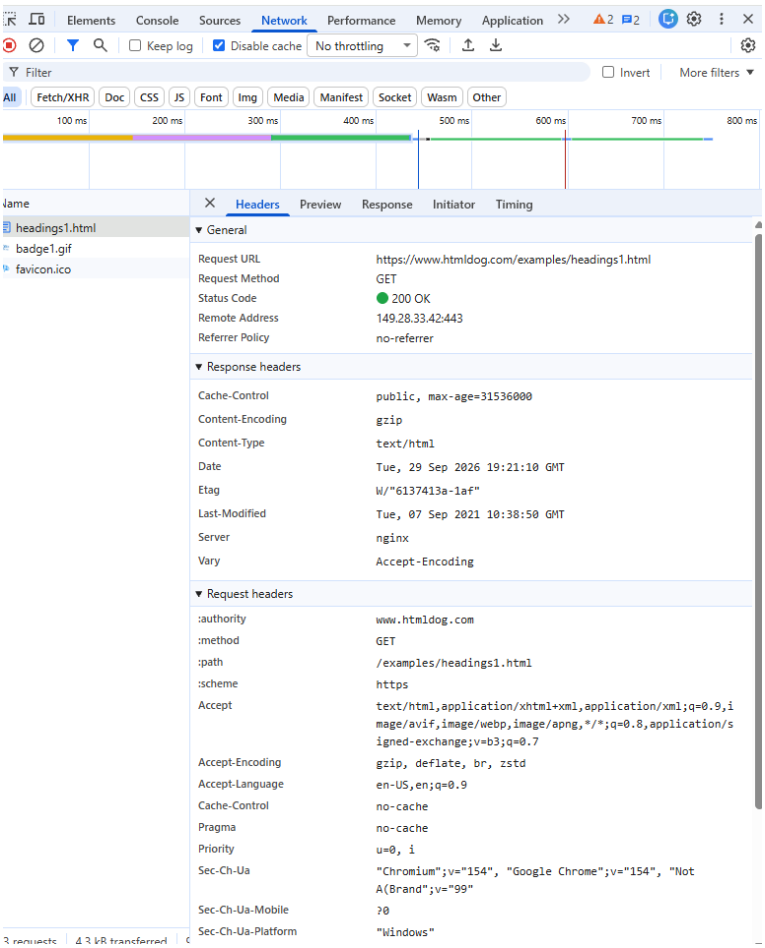
URL: https://www.bol.com/nl/p/hoe-werkt-dat-nou/9200000057347012/?country=BE&suggestionType=browse#product_alternatives

- Protocol: https (HTTP over een beveiligde verbinding)
- Subdomein: www
- Domeinnaam: bol.com (topleveldomein .com)
- Host: www.bol.com (DNS vertaalt deze naam naar een IP-adres)
- Poort: niet vermeld, dus standaardpoort 443 voor https
- Pad: /nl/p/hoe-werkt-dat-nou/9200000057347012/
- Parameters (query string): country=BE en suggestionType=browse
- Fragment: #product_alternatives (een plaats binnen de pagina; wordt niet naar de server gestuurd)

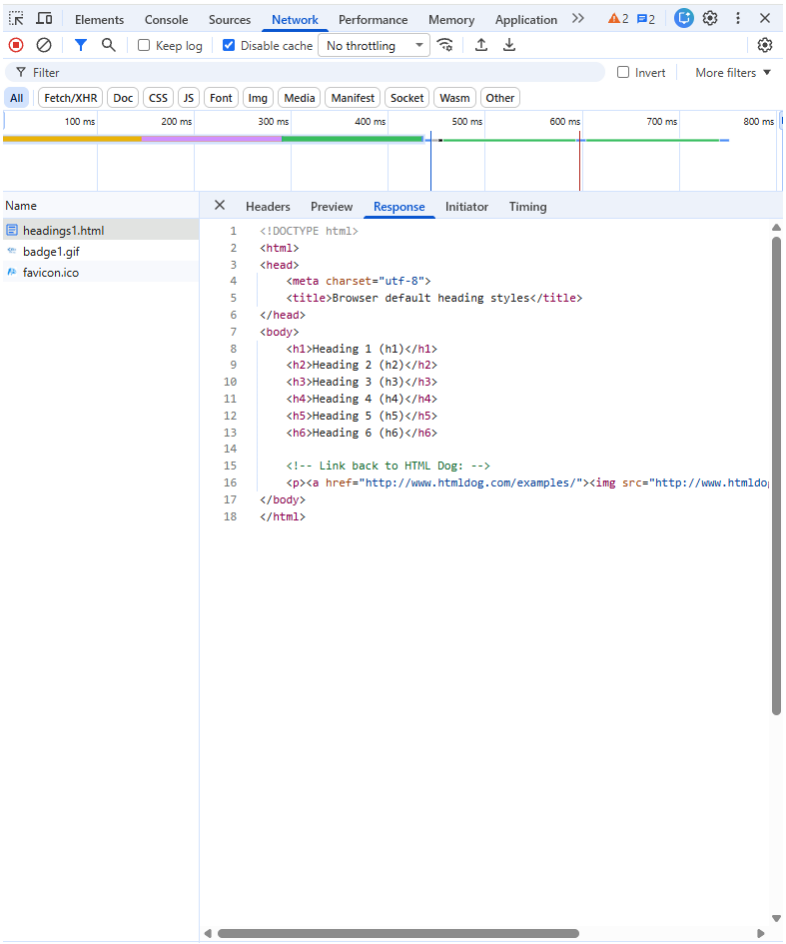
Opdracht 3

Ik opende de Developer Tools (F12), ging naar het tabblad Network en surfte naar https://www.htmldog.com/examples/headings1.html. Om zeker een volledig antwoord van de server te krijgen vinkte ik "Disable cache" aan.

- De browser stuurde een GET-request voor /examples/headings1.html naar www.htmldog.com (IP-adres 149.28.33.42, poort 443).
- De server antwoordde met statuscode 200 OK: de resource werd gevonden en teruggestuurd.
- Request headers (meta-informatie van de browser): bv. Accept (welke formaten de browser aanvaardt), Accept-Language (en-US) en User-Agent (welke browser en welk besturingssysteem).
- Response headers (meta-informatie van de server): bv. Content-Type: text/html, Last-Modified (07 sep 2021), Server: nginx en Cache-Control.
- De response body bevat de HTML-code van de pagina: zes koppen (h1 tot h6) en onderaan een link met een afbeelding naar HTML Dog.



Figuur 2 – Headers van het request voor headings1.html (status 200 OK)



Figuur 3 – Response body: de HTML-code van headings1.html

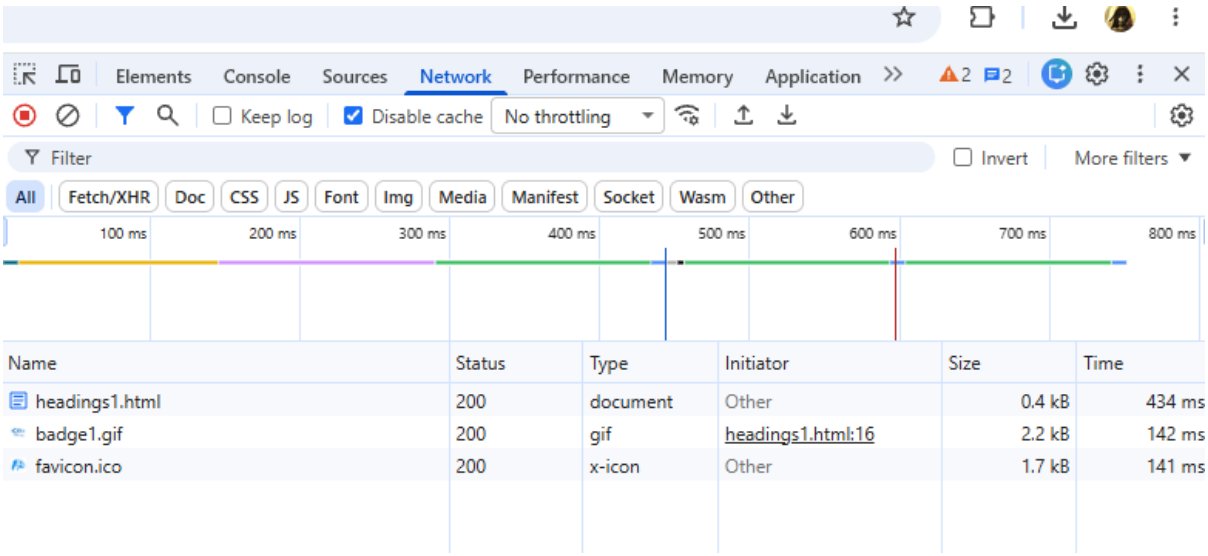
Opdracht 4

Naast headings1.html vroeg de browser nog 2 resources op, in totaal 3 requests:

- badge1.gif (type gif): het logo van HTML Dog onderaan de pagina.
- favicon.ico (type x-icon): het icoontje in het tabblad van de browser.

Dit zie je in het Network-tabblad: elke rij is een apart request. In de kolom Initiator staat waarom het request gebeurde.

- Voor badge1.gif staat als initiator "headings1.html:16". In de response body staat op regel 16 een <img src="http://www.htmldog.com/badge1.gif">-tag. De browser las de HTML, vond deze verwijzing en vroeg de afbeelding daarom apart op. Omdat de afbeelding met http:// en niet met https:// gevraagd werd, gaf de console een "Mixed Content"-waarschuwing en zette de browser het request automatisch om naar HTTPS.
- favicon.ico staat niet in de HTML (initiator "Other"): de browser vraagt dit bestand zelf op om het icoon in het tabblad te tonen.



Figuur 4 – De 3 requests bij het laden van headings1.html, met de kolom Initiator

Opdracht 5

Ik surfte naar https://www.vives.be met het Network-tabblad open ("Disable cache" aangevinkt) en voegde de kolom Domain toe (rechtsklik op de hoofding van de tabel).

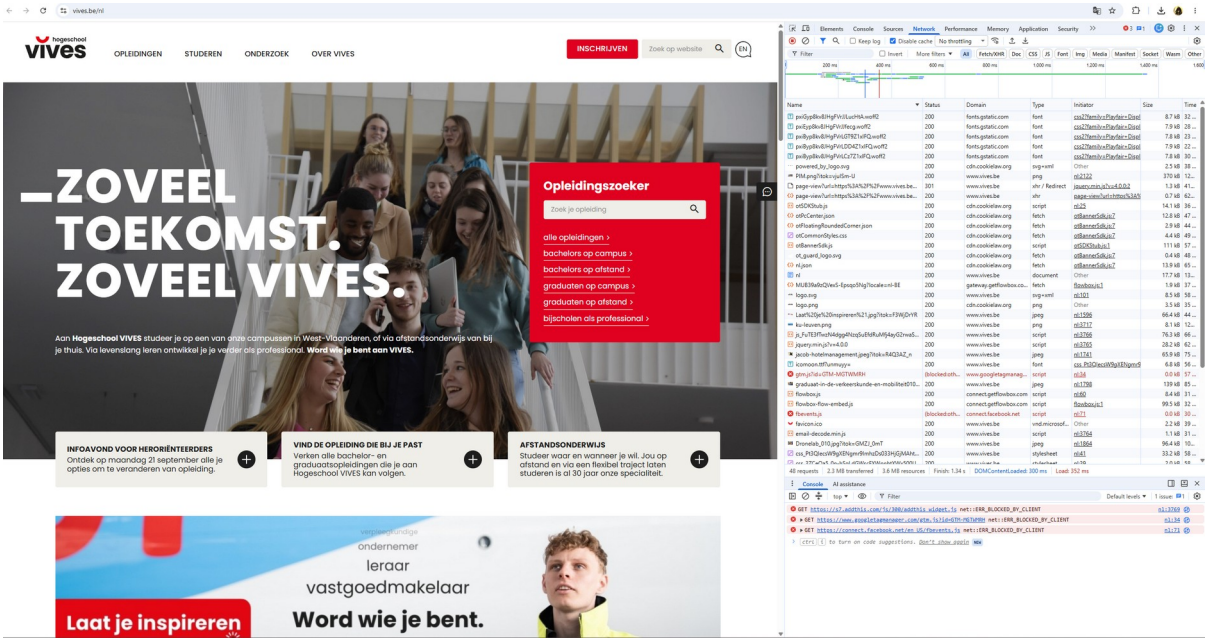
Soorten resources (kolom Type): document (de HTML-pagina "nl"), stylesheets (CSS), scripts (JavaScript), lettertypes (font, .woff2/.ttf), afbeeldingen (jpeg, png, svg), het favicon, en fetch/xhr-requests (data die JavaScript achteraf ophaalt, bv. json-bestanden). Er is ook een redirect (statuscode 301).

Niet alle requests gaan naar dezelfde server. Naast www.vives.be zie je in de kolom Domain onder andere:

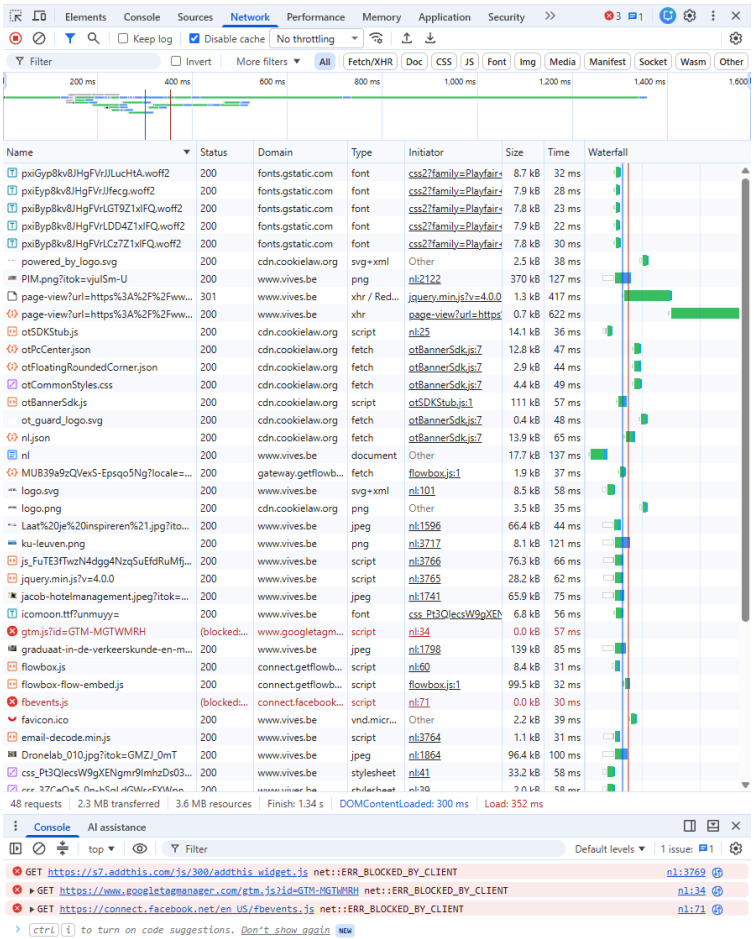
- fonts.gstatic.com (lettertypes van Google Fonts)
- cdn.cookielaw.org (de cookiebanner)
- gateway.getflowbox.com en connect.getflowbox.com (een social media-widget)
- www.googletagmanager.com, connect.facebook.net en s7.addthis.com (tracking en een deel-widget; deze requests zijn "blocked", ze werden tegengehouden door een extensie in mijn browser)

Timing: in de kolom Waterfall stelt elk balkje één request voor, van het versturen tot het ontvangen van de response. Veel balkjes beginnen op (bijna) hetzelfde moment en lopen naast elkaar, bv. de afbeeldingen en scripts van www.vives.be. De browser verstuurt dus een volgend request nog voor het antwoord op het vorige binnen is: de requests gebeuren parallel. Daardoor laadt de hele pagina (48 requests) in ongeveer 1,3 seconden.

Is het aantal gelijktijdige requests beperkt? Ja. Bij verschillende requests van www.vives.be (bv. PIM.png, ku-leuven.png, de jpeg-afbeeldingen) zie je vóór het gekleurde balkje een wit/grijs stuk: dat is wachttijd (Queueing/Stalled). Het request staat klaar maar kan nog niet vertrekken omdat er al te veel requests onafgewerkt zijn. Welke limiet geldt, hangt af van het protocol (zichtbaar via de kolom Protocol in het Network-tabblad): bij HTTP/1.1 opent Chrome maximaal 6 verbindingen tegelijk per domein; bij HTTP/2 (dat grote websites meestal gebruiken) lopen meerdere requests tegelijk over één verbinding, maar ook daar legt de server een maximum op. Omdat de pagina ook resources van andere domeinen haalt, lopen er in totaal toch veel requests tegelijk.



Figuur 5 – www.vives.be met het Network-tabblad en de kolom Domain



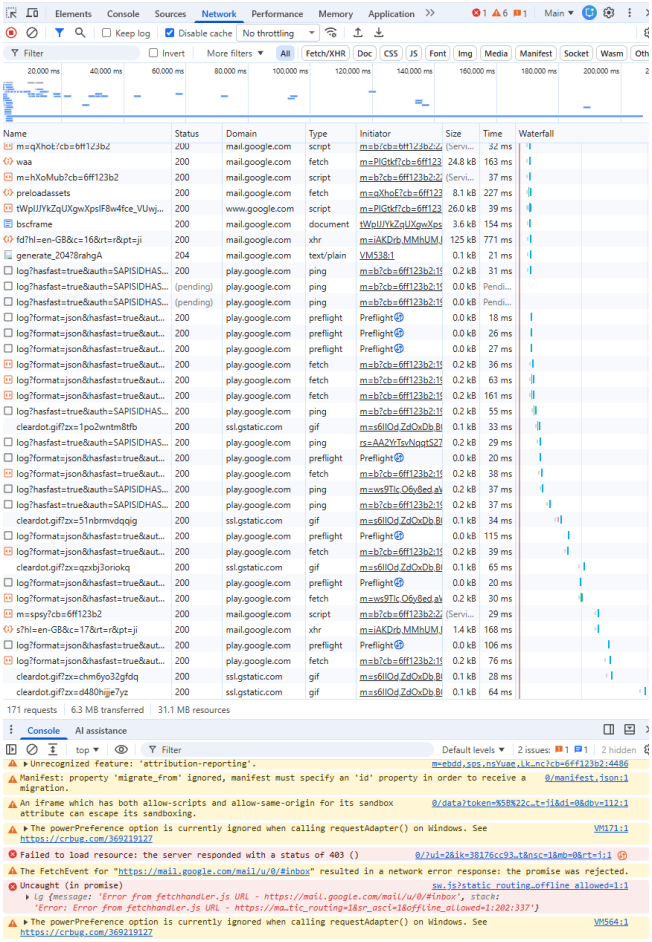
Figuur 6 – Kolom Waterfall: parallelle requests en wachttijd

Opdracht 6

Ik opende het Network-tabblad, surfte naar Gmail en logde in. Daarna liet ik de pagina ongeveer 3 minuten met rust, zonder de muis over de pagina te bewegen. De tijdlijn bovenaan loopt tot meer dan 200 000 ms en er komen regelmatig nieuwe requests bij, ook al verandert er niets op de pagina. In totaal waren er 171 requests.

De spontane requests dienen onder andere voor:

- Nieuwe mails ophalen: requests naar mail.google.com van het type xhr (bv. "s?hl=en-GB..." en "fd?hl=en-GB..."). JavaScript vraagt op de achtergrond aan de server of er nieuwe berichten zijn, zodat die verschijnen zonder dat je de pagina hoeft te herladen.
- Controleren of je nog online bent: kleine afbeeldingen "cleardot.gif" van ssl.gstatic.com (0,1 kB) die regelmatig opgevraagd worden.
- Statistieken en logging: veel requests "log?format=json..." naar play.google.com (type ping, fetch en preflight). Daarmee stuurt Google gegevens over het gebruik en de prestaties van de pagina door. De preflight-requests zijn controles die de browser eerst doet omdat de data naar een ander domein gaat.



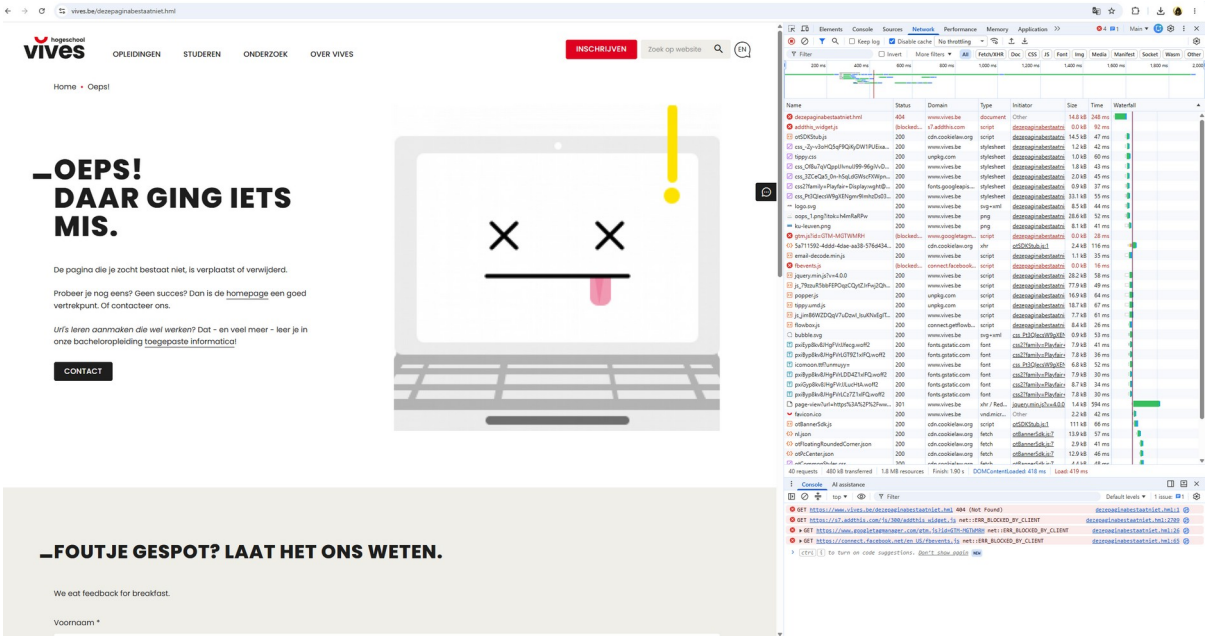
Figuur 7 – Spontane requests in Gmail zonder de pagina aan te raken

Opdracht 7

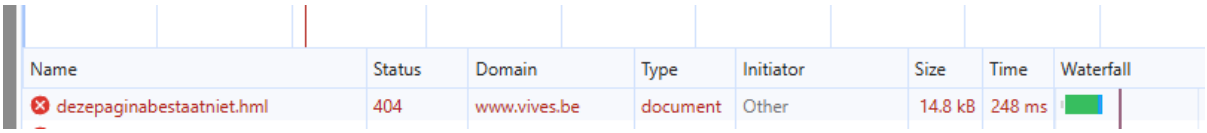
Ik surfte naar https://www.vives.be/dezepaginabestaatniet.hml. Deze pagina bestaat niet, toch kreeg ik een volledige pagina te zien ("Oeps! Daar ging iets mis.") met het menu en de stijl van VIVES.

Hoe komt dit? Het domein www.vives.be bestaat wel. De browser vond via DNS het IP-adres van de server, en de webserver van VIVES ontving het request en antwoordde ook. Alleen vond de server de gevraagde resource niet. Daarom stuurde de server een response met statuscode 404 en als body een eigen foutpagina in HTML. Die foutpagina laadt zelf ook CSS, afbeeldingen en scripts (40 requests in totaal).

Statuscode 404 (Not Found) betekent: de server is bereikbaar en begrijpt het request, maar de gevraagde resource bestaat niet (of niet meer) op deze server. Het is een fout aan de kant van de client (4xx), bv. een verkeerd getypte URL. Hier zie je dat ook aan de tikfout ".hml" in plaats van ".html".



Figuur 8 – De foutpagina van VIVES



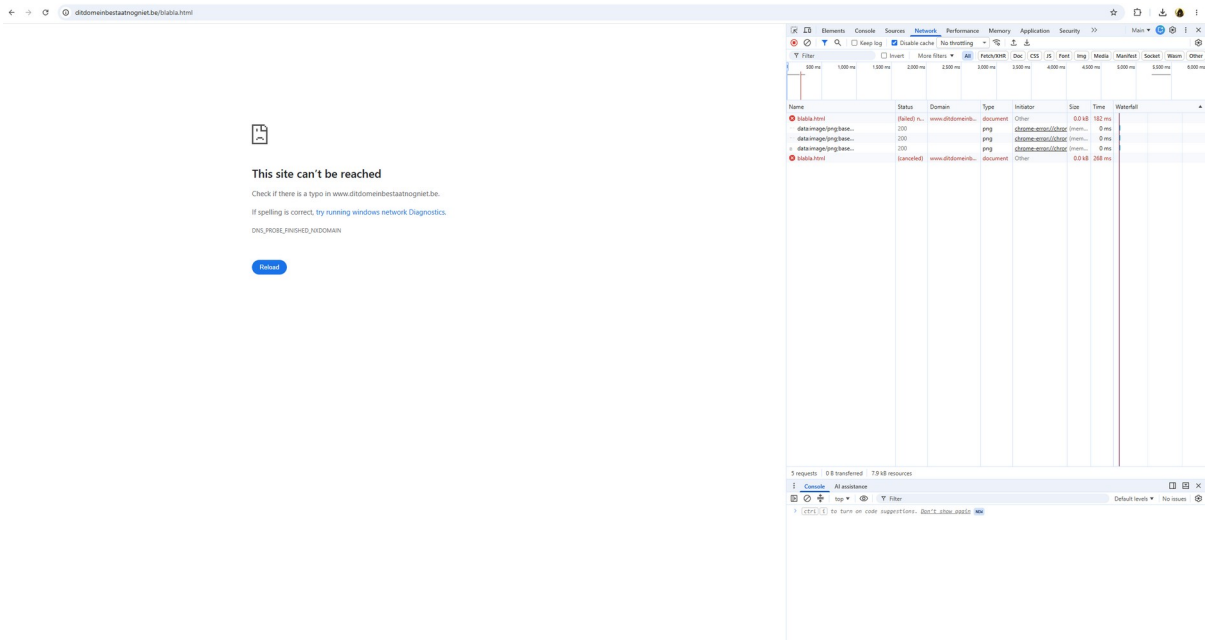
Figuur 9 – Statuscode 404 voor dezepaginabestaatniet.hml

Opdracht 8

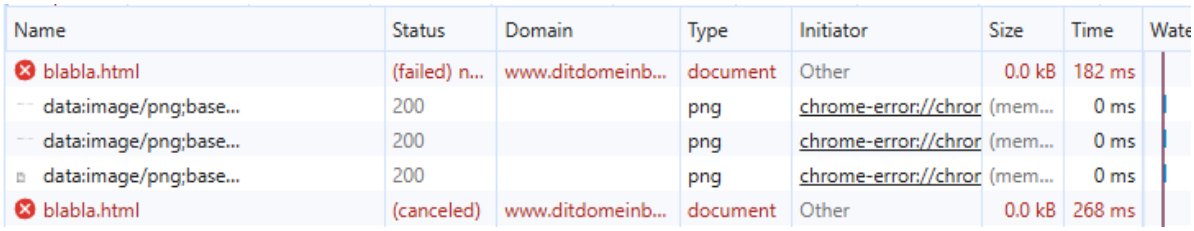
Ik surfte naar https://www.ditdomeinbestaatnogniet.be/blabla.html. Chrome toont nu zijn eigen foutpagina "This site can't be reached" met de foutcode DNS_PROBE_FINISHED_NXDOMAIN.

Verschil met opdracht 7: hier bestaat het domein zelf niet. De DNS-server kon de naam www.ditdomeinbestaatnogniet.be niet vertalen naar een IP-adres (NXDOMAIN = non-existent domain). De browser weet dus niet naar welke server hij het request moet sturen, en er komt geen enkele server aan te pas. In het Network-tabblad staat bij blabla.html geen statuscode maar "(failed)", met 0 kB ontvangen. Er is dus helemaal geen HTTP-response. Er staat nog een tweede rij blabla.html met "(canceled)": Chrome probeerde het request automatisch opnieuw, maar brak die poging af omdat het domein nog steeds niet gevonden werd.

Bij opdracht 7 kwam er wel een response (404) van een bestaande server; bij opdracht 8 is er geen response omdat er geen server gevonden werd. De foutpagina die je ziet, komt hier van de browser zelf en niet van een server (de afbeeldingen erop zijn "data:image/png" uit het geheugen van Chrome).



Figuur 10 – Foutpagina van Chrome: DNS_PROBE_FINISHED_NXDOMAIN



Figuur 11 – Geen statuscode: (failed) en (canceled)

Opdracht 9

2xx – Succes	3xx – Omleiding (redirect)	4xx – Fout van de client	5xx – Fout van de server
200 OK ✔️ gezien Het request is gelukt; de gevraagde resource zit in de body.	**301 Moved Permanently ✔️ gezien** De resource is definitief verplaatst naar een nieuwe URL.	**400 Bad Request** De server begrijpt het request niet (foute syntax).	**500 Internal Server Error** Er ging iets mis op de server zelf.
204 No Content ✔️ gezien Het request is gelukt, maar er is geen inhoud om terug te sturen.	**302 Found** De resource staat tijdelijk op een andere URL.	**401 Unauthorized** Je moet eerst inloggen (authenticatie nodig).	**503 Service Unavailable** De server is tijdelijk overbelast of in onderhoud.
	303 See Other Het antwoord moet met een GET op een andere URL opgehaald worden (bv. na een POST).	**404 Not Found ✔️ gezien** De resource bestaat niet op de server.	

Rood gemarkeerd (✔️ gezien) = statuscodes die ik zelf tegenkwam tijdens dit labo: 200 (bijna alle requests), 204 (generate_204 bij Gmail), 301 (page-view-redirect op vives.be) en 404 (opdracht 7).

Opdracht 10

HTTP request methods: GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS, CONNECT en TRACE.

- GET: een resource opvragen bij de server (bv. een webpagina of afbeelding). De gegevens die je meestuurt, staan in de URL (bv. ?country=BE). Een GET-request mag niets veranderen op de server.
- POST: gegevens naar de server sturen om iets te verwerken of aan te maken, bv. een ingevuld formulier of een login. De gegevens staan in de body van het request.
- PUT / PATCH: een bestaande resource (volledig / gedeeltelijk) aanpassen.
- DELETE: een resource verwijderen.
- HEAD: zoals GET, maar de server stuurt enkel de headers terug, zonder body.
- OPTIONS: vragen welke methods toegelaten zijn (de "preflight"-requests uit opdracht 6 gebruiken OPTIONS).

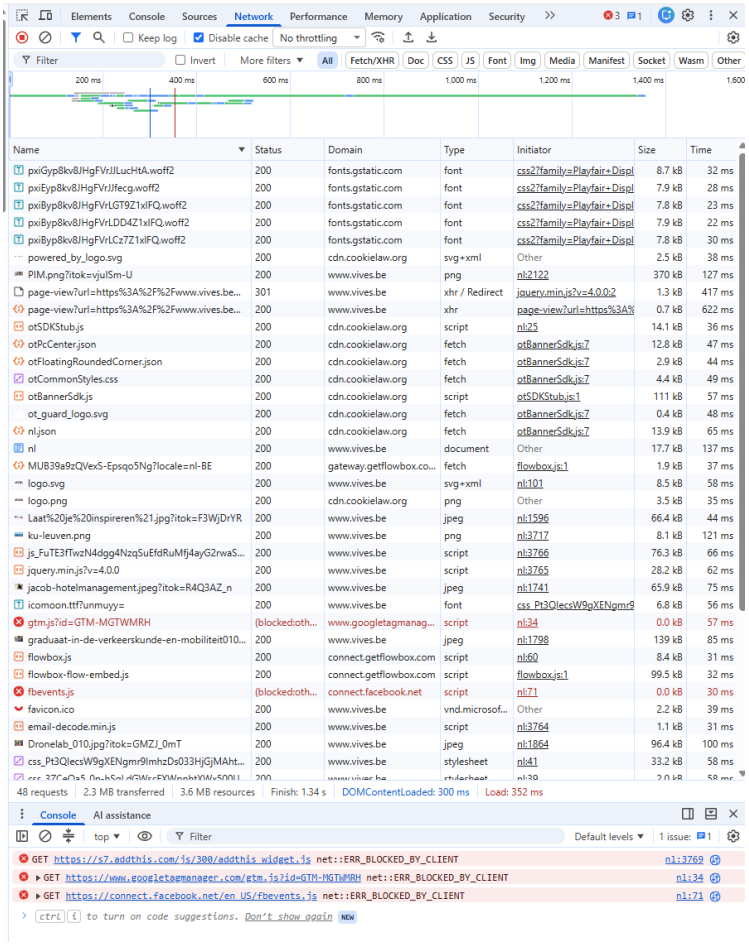
Waar staat de method? In de eerste regel van het request, vóór de URL, bv. "GET /examples/headings1.html HTTP/1.1". In de Developer Tools zie je het bij een request in de tab Headers onder General als "Request Method" (bij opdracht 3: GET), en bij de request headers als ":method: GET". Je kan ook een kolom Method toevoegen aan de tabel (rechtsklik op de hoofding).

Als je een URL in de adresbalk typt en op Enter drukt, gebruikt de browser een GET-request. Ook bij het klikken op een gewone hyperlink wordt een GET-request gebruikt.

De "wis"-link: omdat het wissen gebeurt met een gewone hyperlink, is het een GET-request. Een programma dat alle links automatisch volgt, zoals de Googlebot die pagina's indexeert of een accelerator-plug-in die gelinkte pagina's op voorhand inlaadt, zou dus ook op elke "wis"-link "klikken". Gevolg: alle productgegevens worden gewist, zonder dat iemand dat wilde. Daarom mag een GET-request nooit iets veranderen of verwijderen op de server. Voor het wissen moet je een formulier met POST (of een DELETE-request) gebruiken, liefst met een login en een bevestiging.

Opdracht 11

Onderaan het Network-tabblad staat het totaal: bij het laden van www.vives.be werden 48 requests verstuurd (2,3 MB overgedragen, 3,6 MB aan resources). De pagina was volledig geladen na 1,34 s (DOMContentLoaded na 300 ms, Load na 352 ms).



Figuur 12 – Totaal aantal requests onderaan het Network-tabblad

Opdracht 12

Eerst deed ik op www.vives.be een "Empty cache and hard reload" (rechtsklik op de refresh-knop, met de Developer Tools open). De browser negeert dan zijn cache en vraagt alle resources opnieuw aan de server.

- Verstuurde data: 2,3 MB transferred (48 requests, 3,6 MB aan resources)
- Tijd: Load na 672 ms (DOMContentLoaded na 590 ms), alles klaar (Finish) na 1,43 s

Daarna klikte ik gewoon op de refresh-knop (F5):

- Verstuurde data: nog maar 19,8 kB transferred, meer dan 100 keer minder
- Tijd: Load na 243 ms (DOMContentLoaded na 242 ms), dus bijna 3 keer sneller. De waarde "Finish" (1,51 s) is ongeveer gelijk, omdat er na het laden nog enkele scripts op de achtergrond requests doen.

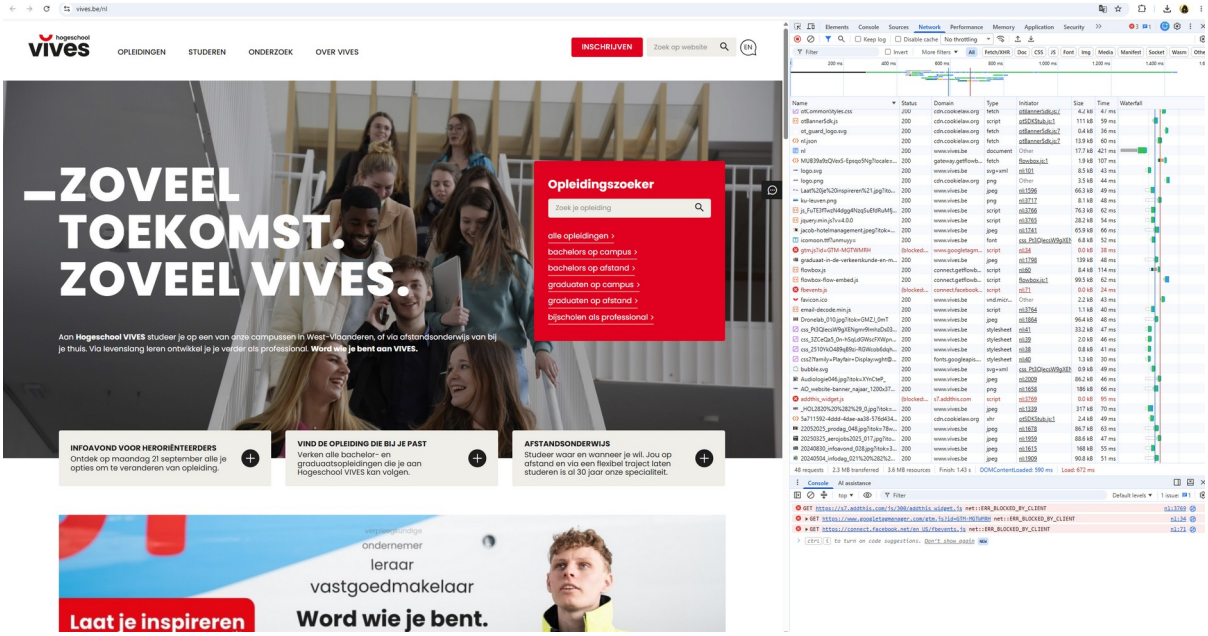
Waarom zoveel minder? De meeste resources werden niet opnieuw bij de server opgevraagd, maar uit de cache van de browser gehaald. Enkel het HTML-document "nl" (17,7 kB) werd echt opnieuw gedownload.

Waar zie je dat? In de kolom Size: bij resources die echt verstuurd werden staat een grootte (bv. 17,7 kB), bij de andere staat "(memory cache)" of "(disk cache)" en een tijd van 0 à 4 ms.

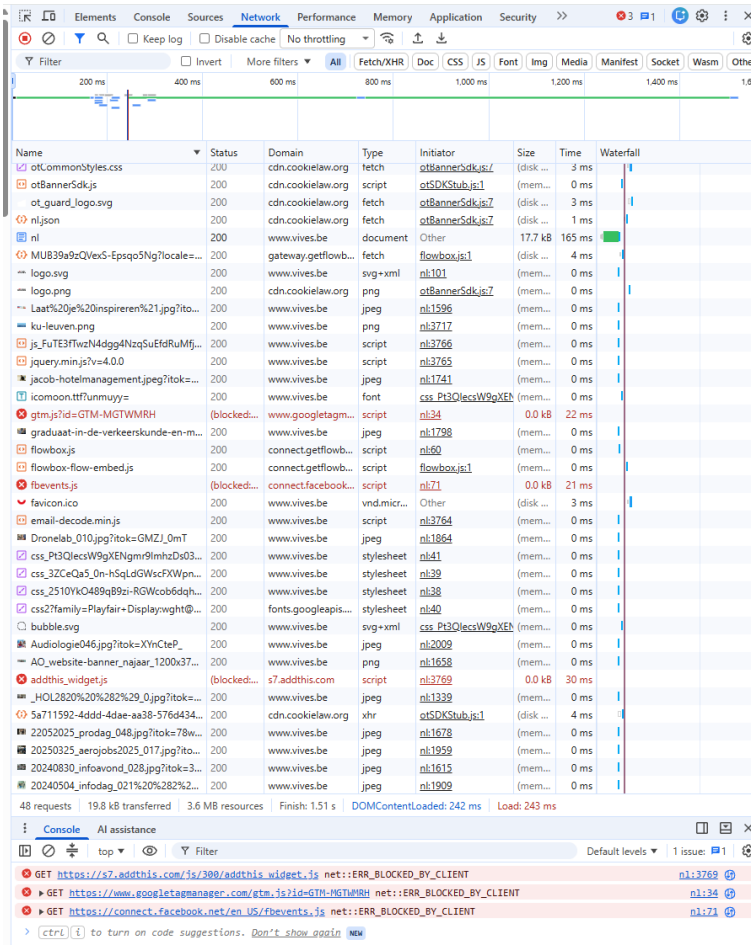
Waar vindt de browser die inhoud? In zijn cache: een kopie in het werkgeheugen (memory cache) of op de harde schijf (disk cache), bewaard bij een vorig bezoek.

Hoe weet de browser hoe lang een resource "vers" is? Via de response headers die de server meestuurt. Bij logo.svg:

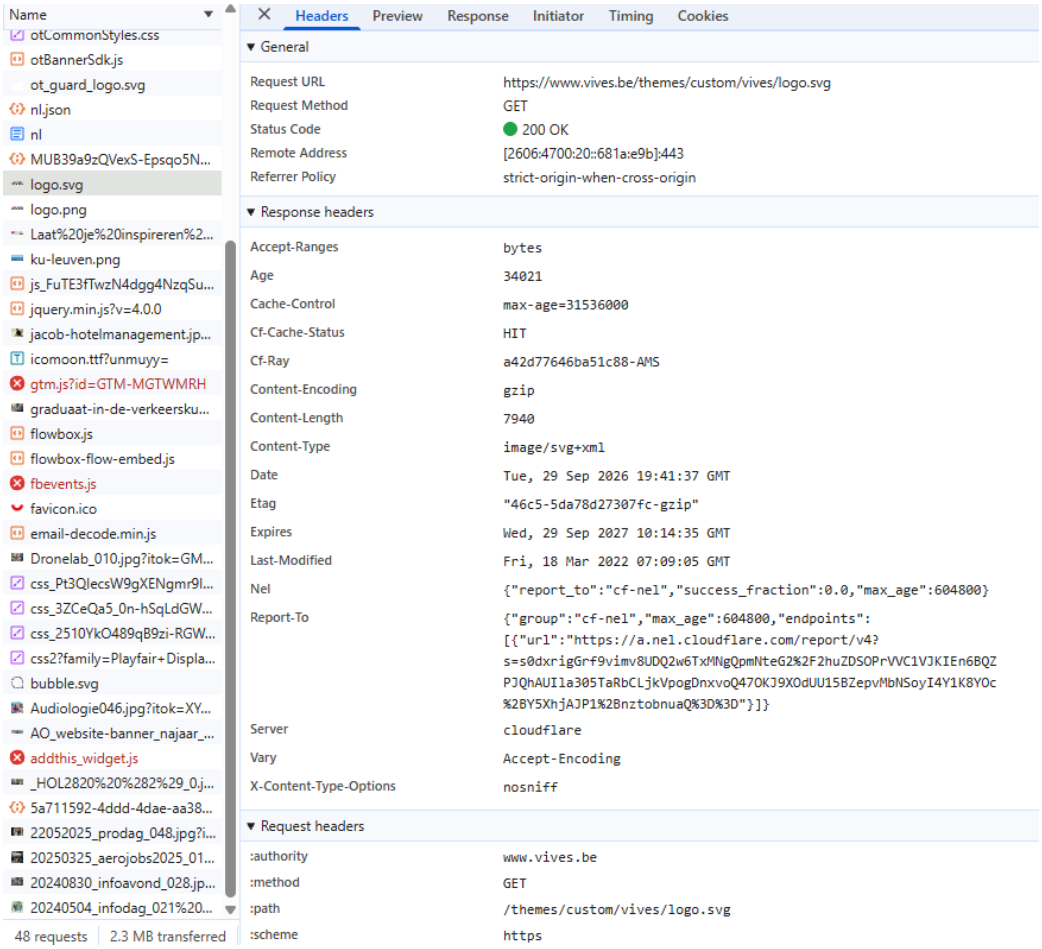
- Cache-Control: max-age=31536000: de browser mag het bestand 31 536 000 seconden (= 1 jaar) gebruiken zonder het opnieuw te vragen.
- Expires: Wed, 29 Sep 2027: tot die datum is het bestand geldig.
- Etag en Last-Modified: als het bestand niet meer vers is, kan de browser met deze waarden aan de server vragen of het gewijzigd is (If-None-Match / If-Modified-Since). Is het niet gewijzigd, dan antwoordt de server met 304 Not Modified zonder body.



Figuur 13 – Na "Empty cache and hard reload": 2,3 MB transferred



Figuur 14 – Na een gewone refresh: 19,8 kB transferred, (memory cache) / (disk cache)



Figuur 15 – Response headers van logo.svg: Cache-Control, Expires, Etag

Opdracht 13

Ik installeerde de extensie Ghostery in Chrome en bezocht de 4 websites. Na het laden van elke pagina klikte ik op het Ghostery-icoon.

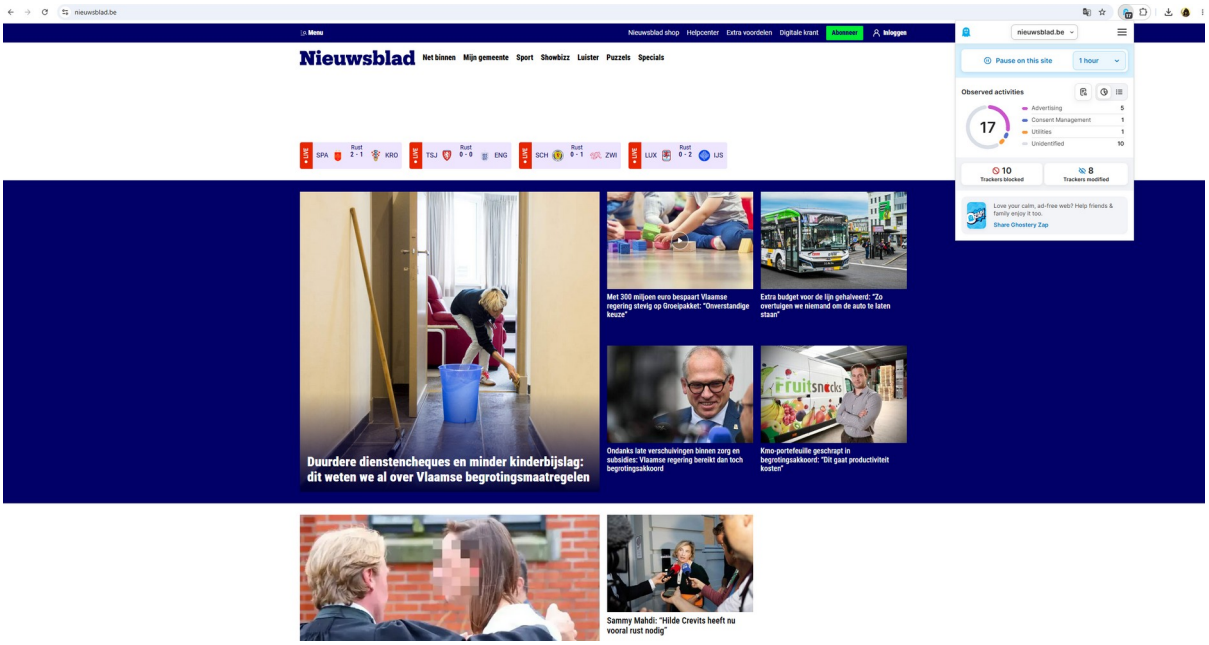
Website	Aantal trackers	Soorten
nieuwsblad.be	**17**	Advertising 5, Consent Management 1, Utilities 1, Unidentified 10 (10 geblokkeerd, 8 aangepast)
cnn.com	**12**	Advertising 4, Site Analytics 1, Consent Management 1, Utilities 4, Unidentified 2 (7 geblokkeerd, 4 aangepast)
vives.be	**7**	Advertising 1, Site Analytics 1, Consent Management 1, Utilities 1, Hosting 2, Other 1 (3 geblokkeerd)
vrt.be/vrtnws	**5**	Advertising 1, Utilities 1, Unidentified 3 (5 geblokkeerd)

Opmerking: bij Nieuwsblad en CNN is "geblokkeerd" + "aangepast" niet gelijk aan het totaal. Ghostery telt die apart: sommige trackers worden zowel (deels) geblokkeerd als aangepast, of worden enkel waargenomen. Het totaal (het getal in de cirkel) is het aantal verschillende trackers.

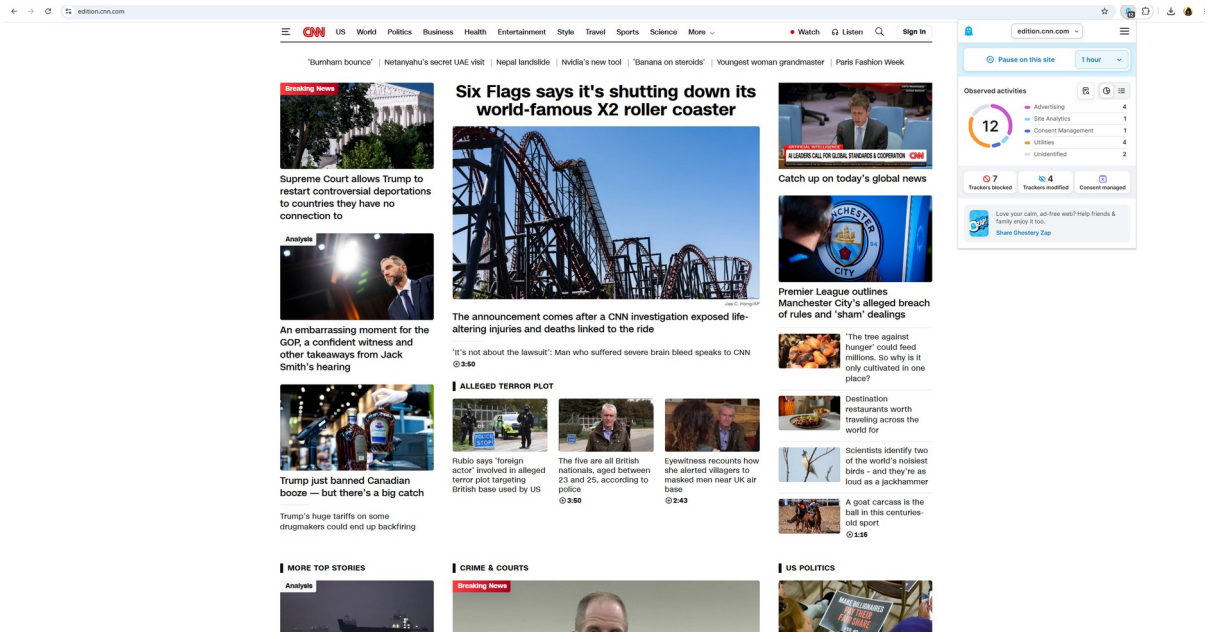
Soorten tracking scripts die ik tegenkwam:

- Advertising: scripts van advertentienetwerken die bijhouden wat je bekijkt, om gerichte reclame te tonen. Op elke site aanwezig, het meest op Nieuwsblad (5) en CNN (4).
- Site Analytics: statistieken over bezoekers (bv. Google Analytics): welke pagina's je bekijkt, hoe lang, met welke browser.
- Consent Management: de cookiebanner die je toestemming vraagt en bijhoudt (bv. cdn.cookielaw.org bij VIVES).
- Utilities: hulpscripts, bv. voor lettertypes, video's of chat.
- Hosting: externe servers die inhoud aanleveren (bv. een CDN).
- Unidentified / Other: trackers die Ghostery niet in een categorie kan plaatsen.

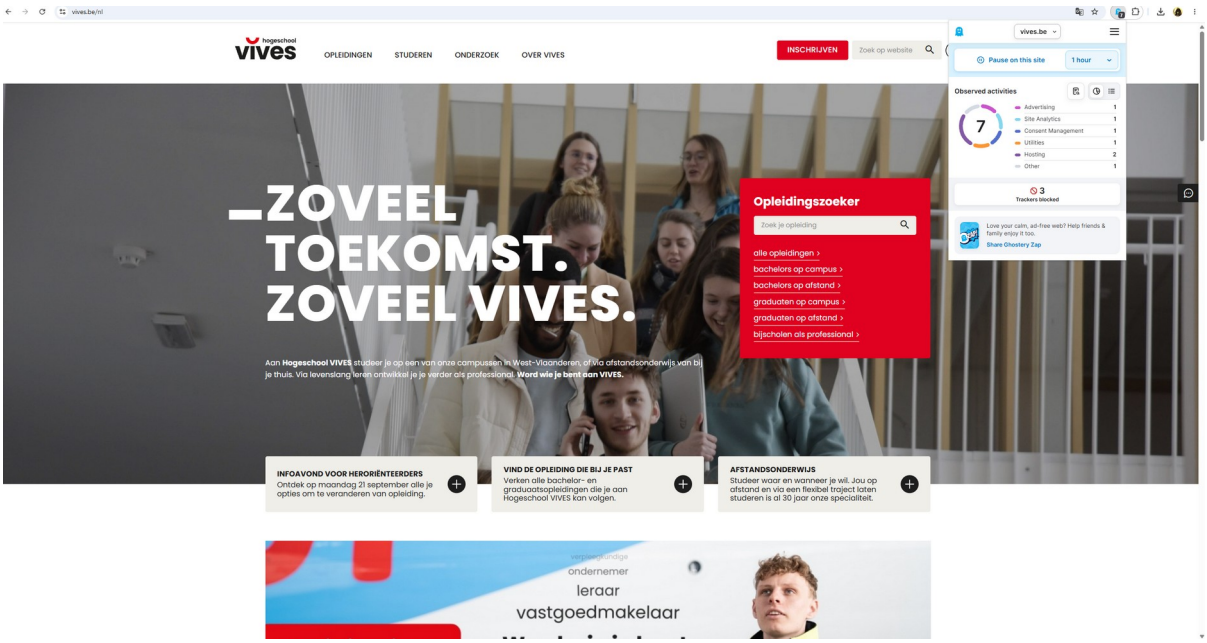
Besluit: nieuwssites met reclame (Nieuwsblad, CNN) bevatten de meeste trackers, omdat ze geld verdienen met advertenties. VIVES (7) en VRT NWS (5) hebben er duidelijk minder.



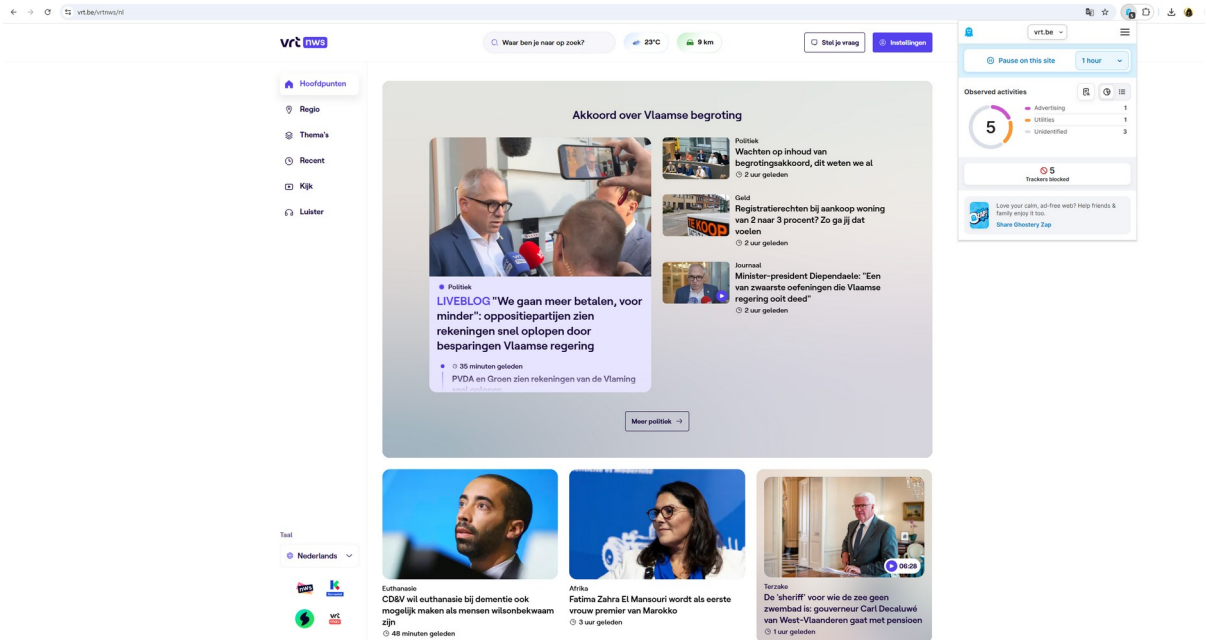
Figuur 16 – Ghostery op nieuwsblad.be: 17 trackers



Figuur 17 – Ghostery op cnn.com: 12 trackers



Figuur 18 – Ghostery op vives.be: 7 trackers



Figuur 19 – Ghostery op vrt.be/vrtnws: 5 trackers

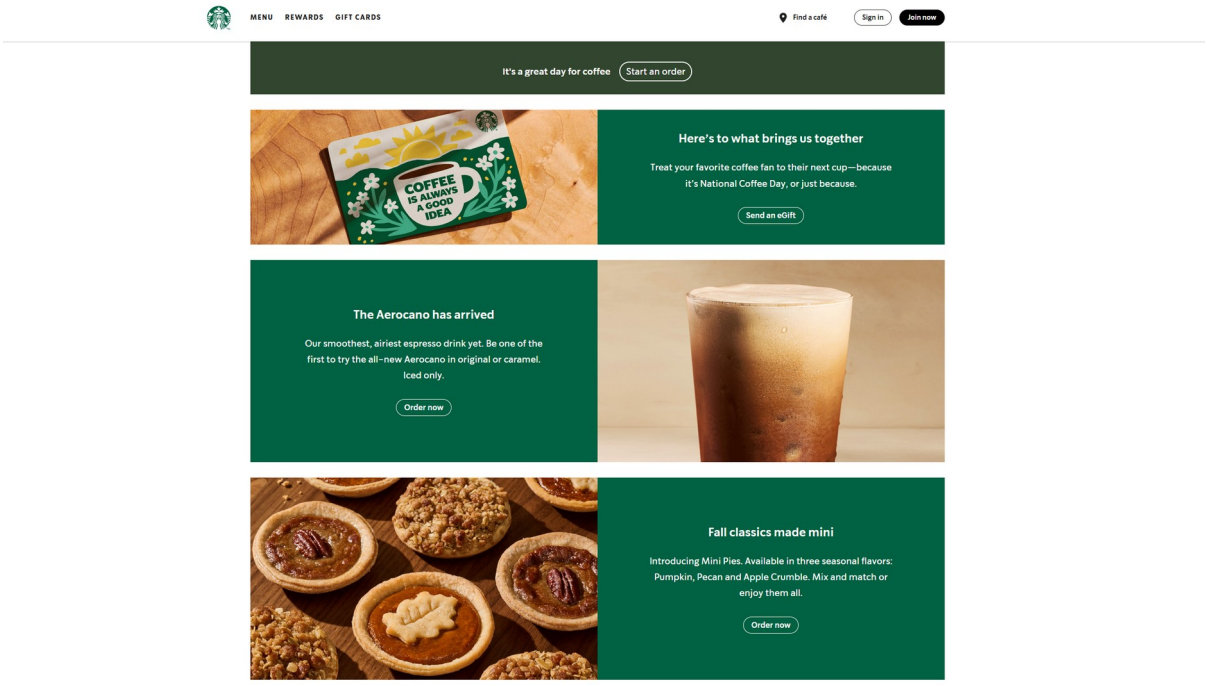
Opdracht 14

Ik opende https://www.starbucks.com in een gemaximaliseerd venster en maakte het venster stap voor stap smaller. De elementen schuiven op en worden kleiner, maar op enkele breedtes verandert de lay-out plots. Dit heet responsive design.

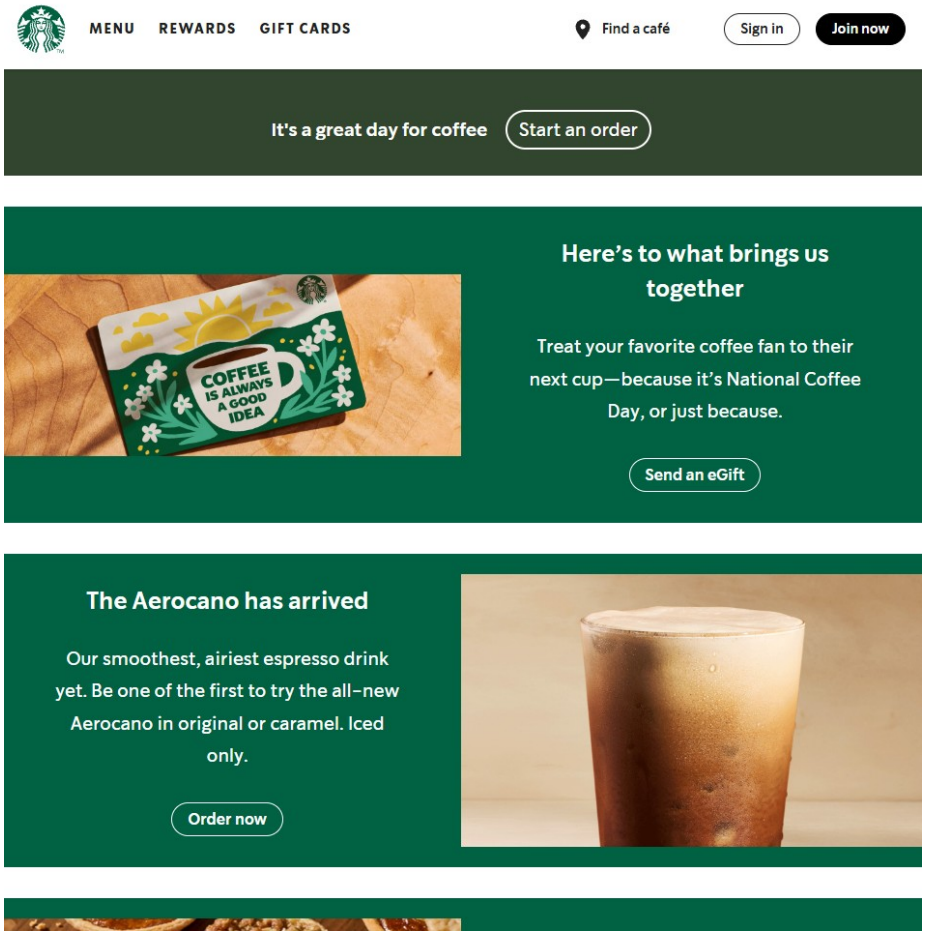
- Lay-out 1 (breed scherm): de inhoud staat in een kader met een vaste maximale breedte, met witruimte links en rechts. Bovenaan een volledige menubalk (MENU, REWARDS, GIFT CARDS, Find a café, Sign in, Join now). Afbeelding en tekst staan naast elkaar in 2 kolommen.
- Lay-out 2 (middelgroot scherm): de witruimte aan de zijkanten verdwijnt en de inhoud vult de volledige breedte van het venster. Het menu is nog volledig zichtbaar en er zijn nog steeds 2 kolommen, maar de tekst loopt over meer regels.
- Lay-out 3 (smal scherm, zoals op een smartphone): de menubalk is vervangen door een hamburgermenu (☰) rechtsboven. De kolommen staan niet meer naast elkaar maar onder elkaar: eerst de afbeelding, daaronder de tekst.

Het laatste screenshot is nog smaller, maar heeft dezelfde lay-out als lay-out 3: de elementen worden enkel kleiner, er is geen abrupte overgang meer.

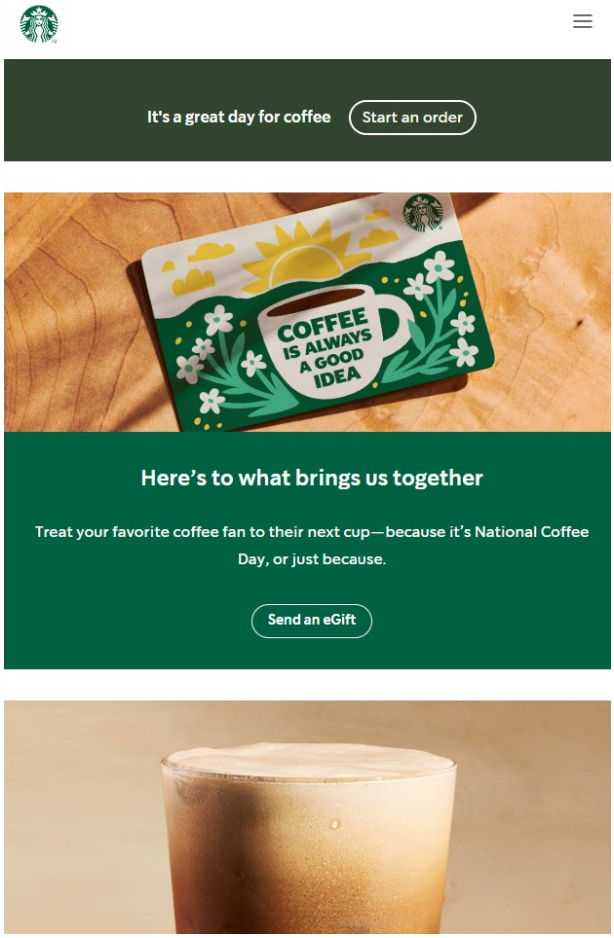
Ik telde dus 3 verschillende lay-outs.



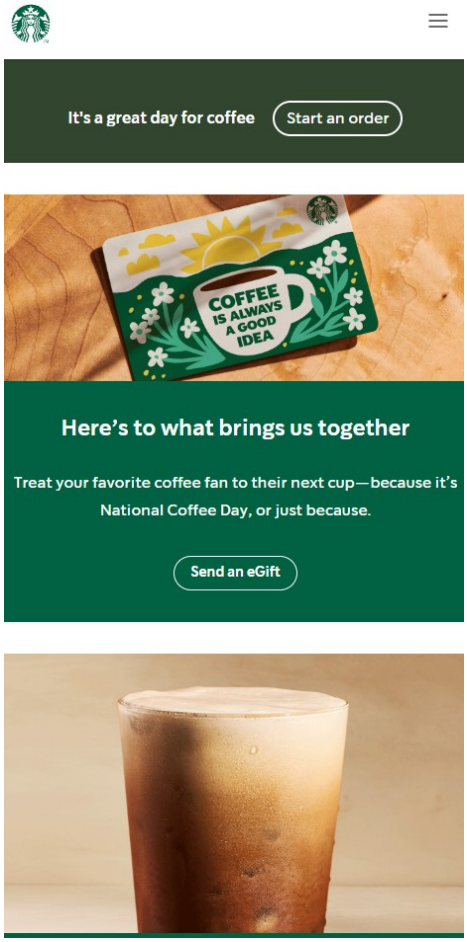
Figuur 20 – Lay-out 1: breed scherm



Figuur 21 – Lay-out 2: middelgroot scherm



Figuur 22 – Lay-out 3: smal scherm met hamburgermenu



Figuur 23 – Nog smaller: nog steeds lay-out 3